Демо 4. Навыки /digest и /cover

Что делаем

Создаём два навыка для дайджест-проекта: `/cover` (генерация обложки через Replicate) и `/digest` (полный цикл выпуска статьи). Настраиваем Stop-хук для валидации статьи перед завершением. Один навык вызывает другой — `/digest` внутри использует `/cover`.

К концу упражнения одна команда `/digest` будет запускать полный цикл: поиск новостей → написание статьи → генерация обложки → коммит → деплой.

Подготовка

1. Репозиторий дайджеста с подключёнными MCP-серверами из Ступени 5:
   - Tavily (для поиска новостей).
   - Replicate (для генерации изображений).
2. Проверьте, что серверы работают:

   ```
   /mcp
   ```

   Оба сервера должны показать статус `connected`.
3. В CLAUDE.md проекта описана редполитика (правила написания статей).
4. jq установлен (`brew install jq`).

Шаг 1. Создаём навык /cover

Навык `/cover` принимает описание статьи и генерирует обложку через Replicate MCP.

Создайте директорию и файл:

```
mkdir -p .claude/skills/cover
```

Содержимое `.claude/skills/cover/SKILL.md`:

```
---
name: cover
description: Generate cover image for a digest article via Replicate
disable-model-invocation: true
argument-hint: [article description]
---

Generate a cover image for the article described below.

Article description: $ARGUMENTS

Steps:
1. Based on the article description, compose a prompt for image
generation.
   Style: modern, minimalist, tech-oriented illustration. No text on the
image.
2. Use the Replicate MCP server to create a prediction with model
   `black-forest-labs/flux-schnell`.
   Input parameters:
   - prompt: your composed prompt in English
   - num_outputs: 1
   - aspect_ratio: "16:9"
3. Download the generated image and save it to the article's directory
   as cover.webp or cover.png.
4. Report the file path of the saved image.
```

Обратите внимание:

Элемент	Зачем
`disable-model-invocation: true`	Навык вызывает внешний API (Replicate). Автоматический запуск нежелателен.
`$ARGUMENTS`	Описание статьи, переданное при вызове: `/cover AI agents reshape enterprise workflow`.

«Use the Replicate MCP server»	Точное имя инструмента зависит от сервера — у `@replicate/mcp-server` одно, у community-обёрток другое (`create_prediction`, `run_model`, `predictions_create`). Claude сам найдёт подходящий инструмент среди подключённых MCP. Посмотреть точное имя — через `/mcp`.

Проверьте вызов:

```
/cover AI breakthrough: new model achieves human-level reasoning on math
benchmarks
```

Claude сгенерирует изображение и сохранит его в проекте.

Шаг 2. Создаём навык /digest

Навык `/digest` — полный цикл выпуска дайджеста: поиск, статья, обложка, коммит, деплой.

```
mkdir -p .claude/skills/digest
```

Содержимое `.claude/skills/digest/SKILL.md`:

```
---
name: digest
description: Full digest article publishing cycle - search, write, cover,
commit, deploy
disable-model-invocation: true
---

Execute a full digest publishing cycle. Follow each step in order.

## Step 1. Search for news
Use the Tavily MCP server to search for the most interesting AI news
from the past week. Search query: "AI news this week".
Pick the single most newsworthy item.

## Step 2. Write the article
Write an article about the selected news item.
Follow the editorial policy from CLAUDE.md strictly:
```

```
- Title, description, publication date, source link.
- The article should be informative and concise.
- Save the article as a markdown file in the content directory
  following the existing naming convention.

## Step 3. Generate cover image
Invoke the /cover skill with a one-sentence description of the article:
/cover <one-sentence description of the article>

## Step 4. Add cover to article
Add the cover image path to the article's frontmatter.

## Step 5. Commit and push
- Stage all new/changed files for the article.
- Commit with message: "feat: add digest — <article title>".
- Push to the remote repository.

## Step 6. Report
Print a summary:
- Article title and file path.
- Cover image path.
- Git commit hash.
- Deployed URL (if known from Vercel).
```

Ключевые решения:

Решение	Причина
`disable-model-invocation: true`	Навык коммитит и пушит. Автоматический запуск недопустим.
Шаг 3 вызывает `/cover`	Навыки могут вызывать другие навыки. `/digest` делегирует генерацию обложки навыку `/cover`.
Пошаговые инструкции	Чёткая последовательность предотвращает пропуск шагов.
Ссылка на CLAUDE.md	Редполитика описана в одном месте. Навык не дублирует правила.

Шаг 3. Создаём Stop-хук для валидации

Stop-хук проверяет, что у статьи есть все обязательные элементы: заголовок, описание, обложка, ссылка на источник. Если чего-то не хватает — Claude не завершает работу.

Создайте файл `.claude/hooks/validate-article.sh`:

```
#!/bin/bash
# Stop-хук: валидация статьи перед завершением.
# Проверяет наличие заголовка, описания, обложки и ссылки на источник.
# Exit 0 = разрешить завершение. Exit 2 = продолжить работу.

# Защита от бесконечного цикла: stop_hook_active.
# При повторном вызове (Claude уже пытался завершить и был заблокирован)
— разрешаем.
ACTIVE=$(jq -r '.stop_hook_active' 2>/dev/null)
if [ "$ACTIVE" = "true" ]; then
  exit 0
fi

# Ищем последний изменённый markdown-файл в content/
# (адаптируйте путь под структуру вашего проекта)
ARTICLE=$(find content/ -name "*.md" -newer .git/index 2>/dev/null | head
-1)

# Если статьи нет — это не цикл публикации, разрешаем завершение
if [ -z "$ARTICLE" ]; then
  exit 0
fi

ERRORS=""

# Проверка: заголовок (строка "title:" в frontmatter)
if ! grep -q "^title:" "$ARTICLE" 2>/dev/null; then
  ERRORS="${ERRORS}Missing title. "
fi

# Проверка: описание (строка "description:" в frontmatter)
if ! grep -q "^description:" "$ARTICLE" 2>/dev/null; then
  ERRORS="${ERRORS}Missing description. "
fi

# Проверка: обложка (строка "cover:" или "image:" в frontmatter)
if ! grep -qE "^(cover|image):" "$ARTICLE" 2>/dev/null; then
  ERRORS="${ERRORS}Missing cover image. "
fi

# Проверка: ссылка на источник (строка "source:" в frontmatter или URL в
тексте)
```

```
if ! grep -qE "(^source:|https?://)" "$ARTICLE" 2>/dev/null; then
  ERRORS="${ERRORS}Missing source link. "
fi

# Если есть ошибки — блокируем завершение
if [ -n "$ERRORS" ]; then
  echo "Article validation failed: ${ERRORS}" >&2
  echo "File: $ARTICLE" >&2
  exit 2
fi

exit 0
```

Сделайте файл исполняемым:

```
chmod +x .claude/hooks/validate-article.sh
```

Разберём ключевой элемент — защиту от бесконечного цикла:

```
ACTIVE=$(jq -r '.stop_hook_active' 2>/dev/null)
if [ "$ACTIVE" = "true" ]; then
  exit 0
fi
```

Без этой проверки: Claude завершает → хук блокирует → Claude пытается исправить → завершает → хук снова блокирует → бесконечный цикл. Поле `stop_hook_active` в JSON на stdin равно `true` при повторном вызове. Guard разрывает цикл: при повторном вызове хук разрешает завершение.

Шаг 4. Добавляем Stop-хук в settings.json

Откройте `.claude/settings.json` и добавьте секцию Stop. Если у вас уже есть хуки из Демо 2, добавьте Stop к существующей конфигурации. Замените `/АБСОЛЮТНЫЙ/ПУТЬ/К/ПРОЕКТУ` на реальный путь.

Полный пример `.claude/settings.json` со всеми хуками (включая Демо 2):

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          {
            "type": "command",
            "command": "jq -r '.tool_input.file_path' | xargs npx
prettier --write 2>/dev/null; exit 0"
          }
        ]
      }
    ],
    "PreToolUse": [
      {
        "matcher": "Bash",
        "hooks": [
          {
            "type": "command",
            "command": "/АБСОЛЮТНЫЙ/ПУТЬ/К/ПРОЕКТУ/.claude/hooks/block-
dangerous.sh"
          }
        ]
      }
    ],
    "Stop": [
      {
        "hooks": [
          {
            "type": "command",
            "command": "/АБСОЛЮТНЫЙ/ПУТЬ/К/
ПРОЕКТУ/.claude/hooks/validate-article.sh"
          }
        ]
      }
    ]
  }
}
```

Обратите внимание: у Stop-хука нет `matcher`. Он сработает при каждом завершении. Guard `stop_hook_active` и проверка наличия статьи обеспечивают, что хук не мешает обычной работе.

Шаг 5. Тестируем полный цикл

Запустите Claude Code и введите:

```
/digest
```

Claude выполнит полный цикл:

1. Найдёт новости через Tavily.
2. Выберет самую интересную.
3. Напишет статью по редполитике из CLAUDE.md.
4. Вызовет `/cover` для генерации обложки.
5. Добавит обложку в frontmatter статьи.
6. Закоммитит и запушит.
7. Покажет отчёт.

Перед завершением Stop-хук проверит статью. Если заголовок, описание, обложка или ссылка на источник отсутствуют — Claude продолжит работу и исправит недостающее.

Навыки вызывают навыки

Ключевая идея этого демо: навыки могут вызывать друг друга. В Шаге 3 инструкции `/digest` стоит:

```
Invoke the /cover skill with a one-sentence description of the article:
/cover <one-sentence description>
```

Claude увидит эту инструкцию, вызовет `/cover` как вложенный навык, получит результат (путь к изображению) и продолжит выполнение `/digest`. Это позволяет строить цепочки навыков — каждый отвечает за свою часть работы.

Чеклист

- [ ] Файл `.claude/skills/cover/SKILL.md` существует с frontmatter и инструкцией.

- [ ] Файл `.claude/skills/digest/SKILL.md` существует с frontmatter и инструкцией.
- [ ] Оба навыка имеют `disable-model-invocation: true`.
- [ ] `/cover` генерирует обложку через Replicate MCP.
- [ ] `/digest` выполняет полный цикл: поиск → статья → обложка → коммит → деплой.
- [ ] `/digest` вызывает `/cover` внутри — навыки связаны.
- [ ] Файл `.claude/hooks/validate-article.sh` существует и исполняемый.
- [ ] Скрипт проверяет: title, description, cover, source link.
- [ ] Guard `stop_hook_active` защищает от бесконечного цикла.
- [ ] Stop-хук добавлен в `.claude/settings.json`.
- [ ] `/hooks` показывает Stop-хук с источником [Project].
- [ ] Статья опубликована на Vercel с заголовком, описанием, обложкой и ссылкой.